Memory Design in Verilog HDL

October 12, 2026

Contents

1 Introduction **2**

2 Memory Organization and Capacity **2**

3 Declaring a Memory in Verilog **3**
3.1 Parameterized memory declaration 3

4 ROM Design **3**
4.1 ROM using a case statement 3
4.2 ROM using an array and initialization 4

5 Loading Memory Contents from a File **4**
5.1 ROM with `$readmemh` 4

6 Single-Port RAM with Synchronous Write **5**

7 Single-Port RAM with Synchronous Read **6**

8 RAM with Reset for the Output Register **7**

9 Memory with a Write Enable **7**

10 Simple Dual-Port RAM **8**

11 Register File **8**

12 Byte-Write Enable **9**

13 Memory Initialization and Unknown Values **9**

14 Self-Checking Testbench for RAM **10**

15 File I/O for Memory Test Vectors and Results **12**

16 Memory Inference and Synthesis Guidelines **12**

17 Common Errors and Best Practices **13**

18 Summary Table **13**

19 Key Takeaways **14**

1 Introduction

Memory is a digital circuit used to store information. In RTL design, memory is commonly described as an array of words. Each word contains a fixed number of bits, and an address selects the word to be accessed.

Memory structures are used in processors, controllers, digital signal processing systems, lookup tables, buffers, FIFOs, and embedded systems.

Common memory types include:

- **ROM (Read-Only Memory):** Stores constant or pre-initialized data.
- **RAM (Random Access Memory):** Supports reading and writing during operation.
- **Single-port RAM:** Uses one address port for memory access.
- **Dual-port RAM:** Provides two access ports, subject to the capabilities of the target memory resource.
- **Register file:** A small collection of words, often with multiple read ports and one or more write ports.

In Verilog, a memory can be declared using a packed vector for each word and an unpacked array of words.

2 Memory Organization and Capacity

A memory is usually described using its depth and width.

- **Depth:** Number of words stored.
- **Width:** Number of bits in each word.
- **Capacity:** Total number of stored bits.
- **Address width:** Number of bits required to select a word.

For a memory with depth D and width W:

$$\text{Capacity in bits} = D \times W$$

If the depth is a power of two, the address width is:

$$A = \log_2(D)$$

For example, a 256×8 memory stores 256 words, each 8 bits wide. Its capacity is 2048 bits, and it requires 8 address bits because $2^8 = 256$.

Memory	Depth	Width	Address bits
16×8	16	8	4
32×16	32	16	5
256×8	256	8	8
1024×32	1024	32	10

If the depth is not a power of two, use enough address bits to represent every valid location. Some binary addresses may then be outside the implemented depth and should be handled deliberately.

3 Declaring a Memory in Verilog

A common Verilog memory declaration is:

```
reg [7:0] mem [0:15];
```

This declares 16 words, each 8 bits wide. The valid word indices are 0 through 15.

The general form is:

```
reg [WIDTH-1:0] mem [0:DEPTH-1];
```

Here, `WIDTH` and `DEPTH` can be parameters. The first range specifies the bits in each word; the second range specifies the array of words.

3.1 Parameterized memory declaration

```
module memory_declaration;
    parameter WIDTH = 8;
    parameter DEPTH = 16;

    reg [WIDTH-1:0] mem [0:DEPTH-1];
endmodule
```

This example demonstrates the declaration only. It does not yet describe read or write behavior.

4 ROM Design

A ROM provides data based on an address. Its contents may be specified directly in the RTL or loaded from a file for simulation or initialization supported by the target tool and device.

4.1 ROM using a case statement

```
module rom16x8 (
    input  wire [3:0] addr,
    output reg  [7:0] data
);
    always @(*) begin
        case (addr)
            4'd0: data = 8'h12;
            4'd1: data = 8'h34;
            4'd2: data = 8'h56;
            4'd3: data = 8'h78;
            4'd4: data = 8'h9A;
```

```
            4'd5: data = 8'hBC;
            4'd6: data = 8'hDE;
            4'd7: data = 8'hF0;
            default: data = 8'h00;
        endcase
    end
endmodule
```

This is a combinational lookup table. The output changes when the address changes. Only addresses 0 through 7 contain nonzero values in this example; the remaining addresses return zero.

4.2 ROM using an array and initialization

```
module rom_array (
    input  wire [3:0] addr,
    output wire [7:0] data
);
    reg [7:0] mem [0:15];

    initial begin
        mem[0] = 8'h12;
        mem[1] = 8'h34;
        mem[2] = 8'h56;
        mem[3] = 8'h78;
        // Initialize other locations as required.
    end

    assign data = mem[addr];
endmodule
```

Important: uninitialized locations may have unknown values in simulation. Whether an initialization construct is supported for hardware implementation depends on the FPGA/ASIC flow. Do not assume that every ASIC synthesis flow will implement an `initial` block as physical ROM contents.

5 Loading Memory Contents from a File

The system tasks `$readmemh` and `$readmemb` can load memory contents from a text file in hexadecimal or binary format, respectively.

5.1 ROM with `$readmemh`

```
module rom_file (
    input  wire [3:0] addr,
    output wire [7:0] data
);
    reg [7:0] mem [0:15];

```

```
    initial begin
        $readmemh("rom_data.hex", mem);
    end

    assign data = mem[addr];
endmodule
```

Example contents of `rom_data.hex`:

```
12
34
56
78
9A
BC
DE
F0
```

Each line represents one hexadecimal word. The file path is interpreted by the simulator or implementation tool, so keep the file in the expected working directory or specify an appropriate path.

For binary input data, use `$readmemb` with a file containing binary digits.

6 Single-Port RAM with Synchronous Write

A single-port RAM uses one address input for read and write access. The following example has synchronous writes and asynchronous reads.

```
module single_port_ram (
    input  wire       clk,
    input  wire       we,
    input  wire [3:0] addr,
    input  wire [7:0] din,
    output wire [7:0] dout
);
    reg [7:0] mem [0:15];

    always @(posedge clk) begin
        if (we)
            mem[addr] <= din;
    end

    assign dout = mem[addr];
endmodule
```

Operation:

- When `we=1`, the selected word is written at the rising clock edge.
- When `we=0`, no memory write occurs.

- The output is combinationally connected to the word selected by `addr`.

This is a useful behavioral description, but the asynchronous read may not match the capabilities or inference templates of every FPGA block RAM. Check the target device and synthesis report.

7 Single-Port RAM with Synchronous Read

Some FPGA block RAMs use synchronous read behavior. In this design, the output register updates at the rising edge of the clock.

```
module sync_read_ram (
    input  wire       clk,
    input  wire       we,
    input  wire [3:0] addr,
    input  wire [7:0] din,
    output reg  [7:0] dout
);
    reg [7:0] mem [0:15];

    always @(posedge clk) begin
        if (we)
            mem[addr] <= din;

        dout <= mem[addr];
    end
endmodule
```

Because nonblocking assignments evaluate their right-hand sides before updating left-hand sides, when a read and write target the same address on one edge, `dout` in this code receives the old memory value. This is commonly called read-first behavior in this coding pattern. Memory primitives and synthesis tools may have different supported read-during-write modes, so confirm the intended behavior for the target.

A write-first variant can explicitly forward the input data:

```
always @(posedge clk) begin
    if (we) begin
        mem[addr] <= din;
        dout <= din;
    end
    else begin
        dout <= mem[addr];
    end
end
```

This code describes write forwarding at the output. Check the exact read-during-write requirements before using it in a design.

8 RAM with Reset for the Output Register

A reset can initialize control or output registers. Resetting every memory word may prevent block RAM inference in some tools or produce a large register-based implementation. The following example resets only the output register.

```
module ram_output_reset (
    input  wire       clk,
    input  wire       rst,
    input  wire       we,
    input  wire [3:0] addr,
    input  wire [7:0] din,
    output reg  [7:0] dout
);
    reg [7:0] mem [0:15];

    always @(posedge clk) begin
        if (rst) begin
            dout <= 8'h00;
        end
        else begin
            if (we)
                mem[addr] <= din;

            dout <= mem[addr];
        end
    end
endmodule
```

The memory contents are not cleared by this reset. If the application requires memory initialization, implement it using a target-supported initialization method or an explicit write sequence.

9 Memory with a Write Enable

A write enable prevents unintended updates. If `we` is low, the stored word remains unchanged.

```
always @(posedge clk) begin
    if (we)
        mem[addr] <= din;
end
```

This is a clocked memory write. Avoid assigning the same memory array from multiple independent procedural blocks unless the target architecture and coding method explicitly support the intended multi-port memory.

10 Simple Dual-Port RAM

A simple dual-port RAM has one write port and one read port. This example writes synchronously and reads synchronously through a registered output.

```
module simple_dual_port_ram (
    input  wire       clk,
    input  wire       we,
    input  wire [3:0] waddr,
    input  wire [7:0] din,
    input  wire [3:0] raddr,
    output reg  [7:0] dout
);
    reg [7:0] mem [0:15];

    always @(posedge clk) begin
        if (we)
            mem[waddr] <= din;

        dout <= mem[raddr];
    end
endmodule
```

The read and write addresses are independent. If a read and write access the same address on the same clock edge, the returned value depends on the coding semantics and target memory's read-during-write mode. Verify this behavior in simulation and against the target memory documentation.

11 Register File

A register file is a small array of words with one write port and multiple combinational read ports. This example uses sixteen 8-bit registers, one write port, and two read ports.

```
module register_file (
    input  wire       clk,
    input  wire       we,
    input  wire [3:0] waddr,
    input  wire [3:0] raddr1,
    input  wire [3:0] raddr2,
    input  wire [7:0] wdata,
    output wire [7:0] rdata1,
    output wire [7:0] rdata2
);
    reg [7:0] regs [0:15];
    always @(posedge clk) begin
        if (we)
            regs[waddr] <= wdata;
    end
    assign rdata1 = regs[raddr1];
    assign rdata2 = regs[raddr2];
endmodule
```

This description has asynchronous reads and synchronous writes. Depending on the target and size, synthesis may implement it using registers and multiplexers rather than dedicated RAM blocks.

12 Byte-Write Enable

For a wider memory word, separate write-enable signals can control individual bytes. The following example writes a 32-bit word in four 8-bit lanes.

```
module byte_enable_ram (
    input  wire        clk,
    input  wire        we,
    input  wire [3:0]  byte_en,
    input  wire [3:0]  addr,
    input  wire [31:0] din,
    output wire [31:0] dout
);
    reg [31:0] mem [0:15];
    integer i;

    always @(posedge clk) begin
        if (we) begin
            for (i = 0; i < 4; i = i + 1) begin
                if (byte_en[i])
                    mem[addr][8*i +: 8] <= din[8*i +: 8];
            end
        end
    end

    assign dout = mem[addr];
endmodule
```

The indexed part-select `[8*i +: 8]` selects an 8-bit lane. The loop has a fixed bound and is intended to describe parallel byte-enable logic, not four clock cycles. Check whether the target memory primitive supports byte write enables.

13 Memory Initialization and Unknown Values

In simulation, memory locations that have not been written or initialized typically contain `X` values. This can help expose bugs rather than silently assuming that memory starts at zero.

Common initialization approaches include:

- Assigning initial values in an `initial` block.
- Loading values with `$readmemh` or `$readmemb`.
- Writing known values through the memory interface during reset or startup.

The supported initialization approach depends on the simulator, synthesis tool, and target technology. For ASIC flows, do not assume that simulation initialization automatically creates a physical ROM or initializes SRAM cells.

14 Self-Checking Testbench for RAM

The following testbench writes two locations and checks the data read back. The example is written for the `single_port_ram` module defined earlier, which has asynchronous reads and synchronous writes.

```
`timescale 1ns/1ps

module single_port_ram_tb;
    reg clk;
    reg we;
    reg [3:0] addr;
    reg [7:0] din;
    wire [7:0] dout;

    integer passed;
    integer failed;

    single_port_ram dut (
        .clk(clk),
        .we(we),
        .addr(addr),
        .din(din),
        .dout(dout)
    );

    always #5 clk = ~clk;

    task check_data;
        input [7:0] expected;
        begin
            #1;
            if (dout === expected) begin
                $display("PASS: addr=%0d data=%h",
                         addr, dout);
                passed = passed + 1;
            end
            else begin
                $display("FAIL: addr=%0d expected=%h actual=%h",
                         addr, expected, dout);
                failed = failed + 1;
            end
        end
    endtask

    initial begin
        clk = 0;
```

```
        we = 0;
        addr = 0;
        din = 0;
        passed = 0;
        failed = 0;

        // Write address 3
        @(negedge clk);
        we = 1;
        addr = 4'd3;
        din = 8'hA5;
        @(negedge clk);
        we = 0;
        check_data(8'hA5);

        // Write address 7
        @(negedge clk);
        we = 1;
        addr = 4'd7;
        din = 8'h3C;
        @(negedge clk);
        we = 0;
        check_data(8'h3C);

        // Confirm address 3 still contains its value
        addr = 4'd3;
        check_data(8'hA5);

        $display("------------------------------");
        $display("Passed = %0d", passed);
        $display("Failed = %0d", failed);
        $display("------------------------------");

        if (failed == 0)
            $display("ALL TESTS PASSED");
        else
            $display("SOME TESTS FAILED");

        $finish;
    end
endmodule
```

This testbench uses a short delay after changing the address so the asynchronous output can settle. A synchronous-read RAM needs a different checker that samples the output after the appropriate clock edge.

15 File I/O for Memory Test Vectors and Results

Test data can be read from a file using `$readmemh` or `$readmemb`. A testbench can also write its results to a log file using `$fopen`, `$fdisplay`, `$fwrite`, and `$fclose`.

```
integer logfile;

initial begin
    logfile = $fopen("memory_results.txt", "w");

    if (logfile == 0) begin
        $display("ERROR: Could not open log file");
        $finish;
    end

    $fdisplay(logfile, "Memory simulation started");
    $fdisplay(logfile, "Address  Expected  Actual");

    // Add test operations and result checks here.

    $fclose(logfile);
end
```

File I/O system tasks are simulation constructs and are not synthesizable hardware. Use them in testbenches, not in synthesizable memory RTL.

16 Memory Inference and Synthesis Guidelines

RTL memory descriptions may map to different hardware resources depending on the target technology and synthesis tool.

- **FPGA block RAM:** Dedicated embedded memory blocks in an FPGA.
- **FPGA distributed RAM:** Memory implemented using lookup tables where supported.
- **Register-based memory:** Flip-flops and selection logic.
- **ASIC SRAM macro:** A hard memory macro instantiated or inferred through a supported design flow.
- **ROM or lookup logic:** Constant data implemented using a memory resource or combinational logic.

Recommended practices:

- Use a clear, supported coding template for the target memory resource.
- Keep memory read and write behavior explicit.
- Avoid resetting every memory location unless required and supported.
- Avoid multiple procedural drivers for the same memory array.
- Check synthesis reports to confirm the inferred resource.

- Verify read latency and read-during-write behavior.
- Confirm memory depth, data width, and address range.
- Use simulation to verify functionality, but use synthesis reports and timing analysis to verify implementation.

17 Common Errors and Best Practices

Common errors include:

- Using an address wider or narrower than intended.
- Accessing an address outside the declared memory range.
- Assuming uninitialized memory contains zero.
- Confusing asynchronous and synchronous read behavior.
- Expecting a specific read-during-write result without defining or checking it.
- Resetting the entire array and unintentionally preventing block RAM inference.
- Using simulation-only system tasks in synthesizable RTL.
- Assuming that a behavioral array will always infer a dedicated memory macro.

18 Summary Table

Construct	**Purpose**
`reg [W-1:0] mem [0:D-1]`	Declares an array of D words, each W bits wide.
`mem[addr]`	Selects a word using its address.
`always @(posedge clk)`	Describes clock-edge-triggered behavior.
`if (we)`	Enables a conditional memory write.
`$readmemh`	Loads memory data from a hexadecimal text file.
`$readmemb`	Loads memory data from a binary text file.
`$fdisplay`	Writes formatted output to a file during simulation.
`parameter`	Makes memory width or depth configurable.

19 Key Takeaways

- Memory in Verilog is commonly described using an array of words.
- Depth specifies the number of words; width specifies the number of bits per word.
- ROM, RAM, register files, and FIFOs have different access and control requirements.
- Synchronous reads and asynchronous reads have different timing behavior.
- Read-during-write behavior must be understood and verified.
- Memory initialization support depends on the target tool and technology.
- Testbenches can use reference models and file I/O to verify memory operations.
- Synthesis reports must be checked to confirm the intended memory resource was inferred.

Key takeaway: A memory RTL description must specify the storage array, address behavior, read and write timing, and initialization assumptions clearly. Correct simulation and synthesis checks are both necessary for reliable memory design.